# NelsonApp / Todo Repuestos Nelson

## Sistema Integral de Gestión para Taller Mecánico y Venta de Repuestos

**NelsonApp** es una solución web progresiva (PWA) de alto rendimiento diseñada específicamente para automatizar, optimizar y centralizar las operaciones de **Todo Repuestos Nelson**. El sistema abarca desde la recepción de vehículos y diagnóstico técnico en taller hasta la venta directa de repuestos, emisión de tickets, control de inventario en tiempo real y seguimiento postventa de clientes.

Construida bajo una arquitectura **Offline-First**, la aplicación garantiza continuidad operativa incluso ante interrupciones en la conectividad a Internet, sincronizando automáticamente los datos con la nube tan pronto como se restablece la red.

---

# Características Principales

## 1. Gestión de Órdenes de Trabajo

- **Flujo completo de servicio:** Creación, asignación, seguimiento de estado (Ingresado, En Diagnóstico, Espera de Repuestos, En Reparación, Finalizado, Entregado) y cancelación.
- **Control de tiempos:** Registro detallado de horas trabajadas por técnico y fechas estimadas de entrega.
- **Presupuestos integrados:** Conversión directa de cotizaciones y diagnósticos en órdenes de trabajo activas.

## 2. Taller y Diagnóstico Técnico

- **Recepciones digitales:** Check-in rápido del vehículo con registro de kilometraje, nivel de combustible y estado de carrocería.
- **Módulo de diagnóstico:** Plantillas de inspección técnica configurable para motores, sistemas eléctricos, frenos y suspensión.
- **Evidencia fotográfica:** Adjuntos visuales de fallas y piezas desgastadas vinculadas directamente a la orden.

## 3. Inventario y Repuestos

- **Catálogo unificado:** Control de repuestos, compatibilidades por marca/modelo/año y equivalencias.
- **Alertas de stock:** Indicadores automáticos de stock mínimo, reabastecimiento crítico y valuación de inventario.
- **Trazabilidad:** Historial de entradas, salidas, ajustes de inventario y asignación de piezas a órdenes de trabajo.

## 4. Venta Directa y Tickets

- **Punto de Venta (POS):** Interfaz ágil para venta en mostrador de repuestos sin necesidad de orden de taller.
- **Emisión de comprobantes:** Generación e impresión instantánea de tickets de venta, remisiones y comprobantes de recepción.
- **Múltiples formas de pago:** Soporte para efectivo, transferencias bancarias, tarjetas y pagos mixtos.

## 5. Cartera de Clientes

- **Historial vehicular:** Expediente clínico interactivo por vehículo (VIN/Placa) con registro de mantenimientos previos y piezas sustituidas.
- **Gestión de contactos:** Perfiles detallados de clientes particulares y corporativos con estados de cuenta y crédito.

## 6. Integración con Supabase y WhatsApp

- **Sincronización Cloud:** Persistencia de datos relacionales, autenticación y storage a través de Supabase.
- **Notificaciones por WhatsApp:** Envío con un clic de actualizaciones de estado, diagnósticos, presupuestos en PDF y alertas de entrega directamente al chat del cliente.

## 7. Copias de Seguridad en Google Drive

- **Respaldos automáticos y manuales:** Exportación de snapshots completos de la base de datos local (IndexedDB) y JSON de respaldo estructurado a Google Drive.
- **Restauración de desastres:** Recuperación ágil del sistema ante fallos de hardware o pérdida de dispositivos.

## 8. PWA Offline-First y Biometría / PIN

- **Funcionamiento Offline:** Service Workers avanzados para caché de aplicación e IndexedDB para almacenamiento local persistente.
- **Seguridad y Acceso Rápido:** Autenticación biométrica (WebAuthn: huella digital / Face ID) o PIN de 4 dígitos para bloqueo rápido de terminal en mostrador o taller.

---

# Estructura del Proyecto

El proyecto está estructurado de forma modular y desacoplada, facilitando el mantenimiento, las pruebas unitarias y la escalabilidad sin dependencias pesadas de frameworks externos./

```
├── css/
│   ├── main.css            # Estilos globales, variables CSS y reset
│   ├── components.css      # Estilos de botones, modales, cards, tablas
│   ├── modules/            # Estilos específicos por módulo
│   └── pwa.css             # Estilos para instalación PWA y modo offline
│
├── js/
│   ├── core/               # Núcleo de la aplicación
│   │   ├── app.js          # Inicializador principal de la PWA
│   │   ├── db.js           # Motor IndexedDB local (Offline engine)
│   │   ├── router.js       # Enrutador cliente SPA
│   │   ├── auth.js         # Lógica de sesión, PIN y Biometría
│   │   └── sync.js         # Motor de sincronización local ↔ Supabase
```

```
|   |
|   ├── modules/            # Lógica de negocio encapsulada
|   |   ├── ordenes.js      # Módulo de Órdenes de Trabajo
|   |   ├── taller.js       # Diagnósticos y recepción vehicular
|   |   ├── inventario.js   # Control de repuestos y stock
|   |   ├── pos.js          # Venta directa y caja
|   |   ├── clientes.js     # Cartera de clientes y vehículos
|   |   └── backup.js       # Integración con Google Drive API
|   |
|   └── libs/               # Librerías locales / Vendored
|       ├── supabase.min.js   # Cliente JS de Supabase
|       └── jspdf.min.js      # Generador de PDF para tickets y presupuestos
|
├── supabase/
|   ├── schema.sql          # Definición de tablas, RLS y triggers
|   └── functions/          # Edge Functions de Supabase (opcional)
|
├── icons/                  # Iconografía PWA y favicons
|   ├── icon-192.png
|   └── icon-512.png
|
```

├── manifest.json # Manifiesto de Aplicación Web WebApp

├── sw.js # Service Worker (Caché y estrategias Offline)

└── index.html # Punto de entrada único (SPA)

---

# Requisitos e Instalación

## Requisitos Previos

- **Servidor Web Local:** Cualquier servidor estático (Live Server de VS Code, Python `http.server`, Nginx, Caddy o Node.js `http-server`).
- **Navegador Moderno:** Chrome, Edge, Safari o Firefox con soporte para Service Workers, WebAuthn e IndexedDB.
- **Cuenta de Supabase:** Para el backend y persistencia remota.
- **Google Cloud Console App:** (Opcional) Para la activación de copias de seguridad en Google Drive.

## Instalación Local

1. **Clonar el repositorio:**

git clone https://github.com/tu-usuario/nelson-app.git

cd nelson-app

2. **Ejecutar en entorno local:**
    - Si utilizas **VS Code**: Haz clic derecho sobre `index.html` y selecciona **Open with Live Server**.
    - Si utilizas **Python**:python3 -m http.server 8080
    - Si utilizas **Node.js**:npx http-server -p 8080
3. Abrir el navegador en `http://localhost:8080`.

---

# Configuración de Variables de Entorno y Base de Datos

## 1. Configuración de Supabase

Crea un proyecto en Supabase y ejecuta el script `supabase/schema.sql` en el **SQL Editor** de tu consola para crear las tablas, políticas de seguridad (RLS) y relaciones.

Crea un archivo de configuración JS o edita el bloque de inicialización en `js/core/db.js`:// Configuración de Supabase en js/core/db.js

const SUPABASE_URL = "https://tu-proyecto.supabase.co";

const SUPABASE_ANON_KEY = "tu-anon-key-aqui";

const supabase = supabase.createClient(SUPABASE_URL, SUPABASE_ANON_KEY);

## 2. Variables de Configuración Globales

| Variable | Descripción | Valor por defecto / Ejemplo |
| --- | --- | --- |
| `SUPABASE_URL` | Endpoint principal de Supabase | `https://xyz.supabase.co` |
| `SUPABASE_ANON_KEY` | Clave pública de API de Supabase | `eyJhbGciOi...` |
| `GOOGLE_CLIENT_ID` | ID de cliente OAuth 2.0 de Google (Drive) | `12345-abc.apps.googleusercontent.com` |
| `WHATSAPP_PREFIX` | Prefijo internacional para mensajes | `+504` |

---

# Despliegue

Al ser un proyecto de arquitectura Frontend estática con Backend-as-a-Service (BaaS), se puede desplegar de forma gratuita y en segundos en plataformas PaaS.

## Despliegue en Netlify

1. Conecta tu repositorio de GitHub / GitLab a Netlify.
2. Configura los parámetros de build:
    - **Build Command:** *(Dejar en blanco)*
    - **Publish Directory:** `.` (Directorio raíz)
3. En la sección **Site Configuration > Environment Variables**, añade tus claves si las inyectas mediante un proceso de build, o asegúrate de que estén configuradas en `js/core/db.js`.
4. Haz clic en **Deploy Site**.

## Despliegue en Vercel

1. Importa el repositorio en el panel de Vercel.
2. Selecciona **Other** como Framework Preset.
3. En **Root Directory**, selecciona `./`.
4. Haz clic en **Deploy**.

**Nota para PWA:** Asegúrate de que el dominio cuente con un certificado SSL (`https://`), ya que los Service Workers y las APIs de Biometría (WebAuthn) requieren un contexto seguro para ejecutarse.

---

# Licencia y Créditos

## Licencia

Este proyecto es software privado y propietario. Todos los derechos están reservados a Todo Repuestos Nelson. Queda prohibida su reproducción, distribución o modificación no autorizada.

## Créditos y Tecnologías Utilizadas

- **Desarrollo y Diseño:** Equipo de Sistemas - Todo Repuestos Nelson.
- **Backend BaaS:** Supabase (PostgreSQL, Auth, Storage).
- **Librerías Client-Side:**
    - JS-PDF - Generación de documentos PDF cliente.
    - Supabase JS Client - Cliente JavaScript oficial.
- **Iconos:** Feather Icons / Material Symbols.